OVERVIEW

This SQL Assignment is the progress that we made so far this semester and is showcasing what we know so far.

1. Creating a table called Customer 2

```sql
CREATE TABLE Customer2 (
    CustomerID INT IDENTITY(100,1),
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    DateOfBirth DATE,
    Address VARCHAR(100),
    City VARCHAR(50),
    State CHAR(2),
    ZipCode VARCHAR(10)
);
```

100 % No issues found

Messages

```
Commands completed successfully.

Completion time: 2026-04-06T18:40:23.1321358-04:00
```

2. Adding a primary key to the table

```sql
ALTER TABLE Customer2
ADD CONSTRAINT PK_Customer2
PRIMARY KEY (CustomerID);
```

3. Defining a Index1 with Primary Key and Index2 with FName and LName

```sql
CREATE UNIQUE INDEX IX_Customer2_CustomerID
ON Customer2 (CustomerID);
go

CREATE INDEX IX_Customer2_Name
ON Customer2 (LastName, FirstName);
go
```

100 % 5 0

Messages

```
Commands completed successfully.

Completion time: 2026-04-06T18:51:55.1387072-04:00
```

4. Inserting 50 values into Customer2 table starting at 101

```sql
SET IDENTITY_INSERT Customer2 ON;

INSERT INTO Customer2 (CustomerID, FirstName, LastName, DateOfBirth, Address, City, State, ZipCode)
VALUES
(101,'John','Smith','1985-01-15','101 Main St','Lansing','MI','48901'),
(102,'Jane','Doe','1990-02-20','102 Oak St','Grand Rapids','MI','49503'),
(103,'Michael','Johnson','1978-03-10','103 Pine St','Detroit','MI','48201'),
(104,'Emily','Davis','1992-04-05','104 Maple St','Ann Arbor','MI','48104'),
(105,'Daniel','Wilson','1983-05-12','105 Cedar St','Flint','MI','48502'),
(106,'Sarah','Taylor','1995-06-18','106 Birch St','Kalamazoo','MI','49001'),
(107,'David','Anderson','1980-07-22','107 Walnut St','Saginaw','MI','48601'),
(108,'Laura','Thomas','1987-08-30','108 Cherry St','Muskegon','MI','49440'),
(109,'James','Jackson','1975-09-14','109 Elm St','Bay City','MI','48706'),
(110,'Linda','White','1993-10-25','110 Spruce St','Midland','MI','48640'),

(111,'Robert','Harris','1982-11-11','111 Ash St','Jackson','MI','49201')
```

100 % 9 0

Messages

```
(50 rows affected)

Completion time: 2026-04-06T18:55:51.2032948-04:00
```

5. Deleting values from Customer2 and deleting the table entirely

```sql
TRUNCATE TABLE Customer2;

DROP TABLE Customer2;
```

100 % 1 0

Messages

```
Commands completed successfully.

Completion time: 2026-04-06T19:11:23.4808352-04:00
```

6. Creates a view of Customer2 with out Date of Birth

```sql
CREATE VIEW vw_Customer2_NoDOB AS
SELECT
    CustomerID,
    FirstName,
    LastName,
    Address,
    City,
    State,
    ZipCode
FROM Customer2;
```

100 % 8 0

Messages

```
Commands completed successfully.

Completion time: 2026-04-06T19:09:18.6712793-04:00
```

7. Shows all values within Customer2 view sorted by First Name

```sql
SELECT *
FROM vw_Customer2_NoDOB
ORDER BY FirstName DESC;
```

100 % 2 0

Results Messages

	CustomerID	FirstName	LastName	Address	City	State	ZipCode
1	117	Thomas	Clark	117 Fir St	St Johns	MI	48879
2	118	Susan	Rodriguez	118 Palm St	Charlotte	MI	48813
3	129	Steven	Hill	129 Peach St	Gaylord	MI	49735
4	146	Stephanie	Collins	146 Poplar St	Rockford	MI	49341
5	106	Sarah	Taylor	106 Birch St	Kalamazoo	MI	49001
6	130	Sandra	Scott	130 Pear St	Cheboygan	MI	49721
7	147	Ryan	Stewart	147 Willow St	Sparta	MI	49345
8	111	Robert	Harris	111 Ash St	Jackson	MI	49201
9	148	Rebecca	Sanchez	148 Hickory St	Cedar Springs	MI	49319
10	131	Paul	Green	131 Plum St	Escanaba	MI	49829
11	112	Patricia	Martin	112 Poplar St	Monroe	MI	48161
12	124	Nancy	Young	124 Alder St	Mt Pleasant	MI	48858
13	140	Michelle	Turner	140 Birch St	Hart	MI	49420
14	103	Michael	Johnson	103 Pine St	Detroit	MI	48201
15	144	Melissa	Evans	144 Spruce St	Grant	MI	49327
16	121	Matthew	Walker	121 Dogwood St	Holland	MI	49423
17	125	Mark	Hernandez	125 Beech St	Big Rapids	MI	49307
18	126	Lisa	King	126 Cottonwood St	Cadillac	MI	49601
19	110	Linda	White	110 Spruce St	Midland	MI	48640
20	108	Laura	Thomas	108 Cherry St	Muskegon	MI	49440

8. IF statements

```sql
    -- 1⃣ Check CustomerID validity
    IF @CustomerID <= 0
    BEGIN
        PRINT 'Invalid CustomerID: must be positive.';
        RETURN;
    END

    -- 2⃣ Check First and Last Name lengths
    IF LEN(@FName) = 0 OR LEN(@LName) = 0
    BEGIN
        PRINT 'First and Last names cannot be empty.';
        RETURN;
    END

    -- 3⃣ Check PhoneNbr if provided
    IF @PhoneNbr IS NOT NULL
    BEGIN
        IF LEN(RTRIM(@PhoneNbr)) <> 10
            PRINT 'PhoneNbr should be exactly 10 characters.';
        ELSE
            PRINT 'PhoneNbr length is valid.';
    END
    ELSE
        PRINT 'PhoneNbr not provided.';

    -- 4⃣ Check PIN format if provided
```

9. If and Else If statements

```sql
    -- 4□ Check PIN format if provided
    IF @PIN IS NOT NULL
    BEGIN
        IF LEN(RTRIM(@PIN)) < 4
            PRINT 'PIN is too short.';
        ELSE IF LEN(RTRIM(@PIN)) = 4
            PRINT 'PIN length is valid.';
        ELSE
            PRINT 'PIN length is longer than expected.';
    END
    ELSE
        PRINT 'PIN not provided.';
END;
```

10. When statements

```sql
SELECT
    CustomerID,
    PIN,
    CASE
        WHEN LEN(RTRIM(PIN)) < 4 THEN 'Short PIN'
        WHEN LEN(RTRIM(PIN)) = 4 THEN 'Valid PIN'
        WHEN LEN(RTRIM(PIN)) > 4 THEN 'Long PIN'
        ELSE 'No PIN'
    END AS PIN_Status
FROM Customer;
```

100 % 6 0

Results Messages

	CustomerID	PIN	PIN_Status
1	98	3256	Valid PIN
2	99	5189	Valid PIN
3	100	NULL	No PIN
4	100	NULL	No PIN
5	101	NULL	No PIN

11. Alter the Customer table to include age

```sql
ALTER TABLE Customer
ADD Age INT NULL;
```

12. Generate and capture Error Log

```sql
CREATE TABLE ErrorLog (
    ErrorID INT IDENTITY(1,1) PRIMARY KEY,
    ErrorMessage NVARCHAR(4000),
    ErrorTime DATETIME DEFAULT GETDATE()
);
go

CREATE PROCEDURE usp_UpdateCustomerPIN
    @CustomerID INT,
    @NewPIN NCHAR(10)
AS
BEGIN
    BEGIN TRY
        -- Example update
        UPDATE Customer
        SET PIN = @NewPIN
        WHERE CustomerID = @CustomerID;

        -- Check if any row was affected
        IF @@ROWCOUNT = 0
            THROW 50001, 'CustomerID not found. Update failed.', 1;

        PRINT 'Customer PIN updated successfully.';
    END TRY
    BEGIN CATCH
        -- Log the error
        INSERT INTO ErrorLog (ErrorMessage)
        VALUES (ERROR_MESSAGE());

        -- Optionally, re-raise the error
        PRINT 'An error occurred and has been logged.';
    END CATCH
END;
go


EXEC usp_UpdateCustomerPIN @CustomerID = 101, @NewPIN = '1234';
go

EXEC usp_UpdateCustomerPIN @CustomerID = 9999, @NewPIN = '5678';
go

SELECT * FROM ErrorLog;
go
```

13. Define and use variables

```sql
-- Declare variables
DECLARE @CustomerID INT;
DECLARE @CustomerPIN NCHAR(10);
DECLARE @Message NVARCHAR(100);

-- Assign values to variables
SET @CustomerID = 101;
SET @CustomerPIN = '1234';
SET @Message = 'Customer PIN update simulation';
```

14. Takes variable checks it for error and then logs it

```sql
CREATE PROCEDURE usp_UpdateCustomerPIN_WithLogging
    @CustomerID INT,
    @NewPIN NCHAR(10)
AS
BEGIN
    BEGIN TRY
        -- Attempt the update
        UPDATE Customer
        SET PIN = @NewPIN
        WHERE CustomerID = @CustomerID;

        -- Check if any row was affected
        IF @@ROWCOUNT = 0
            THROW 50001, 'CustomerID not found. Update failed.', 1;

        PRINT 'Customer PIN updated successfully.';
    END TRY
    BEGIN CATCH
        -- Log the error to ErrorLog
        INSERT INTO ErrorLog (ErrorMessage)
        VALUES (ERROR_MESSAGE());

        -- Return a friendly message
        PRINT 'An error occurred and has been logged.';
    END CATCH
END;
```

15. Incremental data

```sql
CREATE TABLE Cars (
    CarID INT IDENTITY(1,1) PRIMARY KEY,
    Make NVARCHAR(20),
    Model NVARCHAR(20),
    Year INT,
    Color NVARCHAR(20),
    Price DECIMAL(10,2)
);

DECLARE
    @i INT = 1,
    @Make NVARCHAR(20),
    @Model NVARCHAR(20),
    @Year INT = 2015,
    @Color NVARCHAR(20),
    @Price DECIMAL(10,2) = 15000;

WHILE @i <= 10
BEGIN
    -- Assign values dynamically
    SET @Make = 'Make' + CAST(@i AS NVARCHAR(2));
    SET @Model = 'Model' + CAST(@i AS NVARCHAR(2));
    SET @Color = CASE
                    WHEN @i % 3 = 0 THEN 'Red'
                    WHEN @i % 3 = 1 THEN 'Blue'
                    ELSE 'Black'
                END;
    SET @Year = 2015 + @i;            -- Increment year for each car
    SET @Price = 15000 + (@i * 1000); -- Increment price

    -- Insert into Cars table
    INSERT INTO Cars (Make, Model, Year, Color, Price)
    VALUES (@Make, @Model, @Year, @Color, @Price);

    -- Increment counter
    SET @i = @i + 1;
END;
```

16. Create table and removed key duplication restraint

```sql
ALTER TABLE Lake
DROP CONSTRAINT PK_Lake;
GO

CREATE UNIQUE INDEX LakeIDX3
ON Lake(LakeID)
WITH (IGNORE_DUP_KEY = ON);
GO

INSERT INTO Lake
    (LakeID, Name)
VALUES
    (1, 'Michigan'),
    (2, 'Huron'),
    (2, 'Huron'),
    (2, 'Huron'),
    (0, 'Superior'),
    (3, 'Erie'),
    (4, 'Ontario')

SELECT * FROM Lake
```

100 % 7 0

Results Messages

	LakeID	Name
1	0	Superior
2	1	Michigan
3	2	Huron
4	3	Erie
5	4	Ontario

17. Shows duplications of key value

```sql
SELECT  Name, COUNT(*) occurrences
FROM LakeDup
GROUP BY  Name
--HAVING COUNT(*) > 1;
```

100 % 9 0

Results Messages

	Name	occurrences
1	Erie	1
2	Huron	3
3	Michigan	1
4	Ontario	1
5	Superior	1

18. Insert routine for error checking

```sql
DECLARE @ErrorHold int = 0
INSERT INTO LakeDup
    (LakeID, Name)
VALUES
    (1, 'Michigan')
Set @ErrorHold = @@Error
IF @ErrorHold != 0
    .....error routine
```